TOWER

Command Line 101

The command line may seem like a mysterious place, but you actually only need to know a few basic commands to start feeling comfortable.

Table of Contents

Command Line Language: **EN** | CN

Command Line 101

For many non-technical people, the command line (also referred to as CLI, Terminal, bash, or shell) is a place of mystery. However, you only have to know a handful of basic commands to start feeling comfortable.

Opening Your Command Line Interface

find it in the "Applications" folder, inside the "Utilities" subfolder.

On Windows, following the installation guidelines earlier in this book will provide you with an application called "Git Bash". You'll find it in your Windows START menu, inside the "Git" folder.

Finding Your Way Around

As the name already implies, the command line is used to execute commands: you type something and confirm the command by hitting ENTER. Most of these commands are dependent on your current location - where "location" means a certain directory or path on your computer. So, let's issue our first command to find out where we currently are:

```
$ pwd
```

You can easily remember this command when you know what it stands for: "**p**rint **w**orking **d**irectory". It will return the path to a local folder on your computer's disk.

To change this current working directory, you can use the "cd" command (where "cd" stands for "**c**hange **d**irectory"). For example, to move one directory upwards (into the current folder's parent folder), you can just call:

```
$ cd ..
```

To move into subfolders, you would call something like this:

```
$ cd name-of-subfolder/sub-subfolder/
```

/Users/your-username/projects/", you should use this shorthand form:

```
$ cd ~/projects/
```

Also very important is the "ls" command that lists the file contents of a directory. I suggest you always use this command with two additional options: "-l" formats the output list a little more structured and "-a" also lists "hidden" files (which is helpful when working with version control). Showing the contents of the current directory works as follows:

```
$ ls -la
```

Working with Files

The most important file operations can be controlled with just a handful of commands.

Let's start by removing a file:

```
$ rm path/to/file.ext
```

When trying to delete a folder, however, please note that you'll have to add the "-r" flag (which stand for "recursive"):

```
$ rm -r path/to/folder
```

Moving a file is just as simple:

```
$ mv path/to/file.ext different/path/file.ext
```

```
$ mv old-filename.ext new-filename.ext
```

If, instead of moving the file, you want to copy it, simply use "cp" instead of "mv".

Finally, to create a new folder, you call the "make directory" command:

```
$ mkdir new-folder
```

Generating Output

The command line is quite an all-rounder: it can also display a file's contents - although it won't do this as elegantly as your favorite editor. Nonetheless, there are cases where it's handy to use the command line for this. For example when you only want to take a quick look - or when GUI apps are simply not available because you're working on a remote server.

The "cat" command outputs the whole file in one go:

```
$ cat file.ext
```

In a similar way, the "head" command displays the file's *first* 10 lines, while "tail" shows the *last* 10 lines. You can simply scroll up and down in the output like you're used to from other applications.

The "less" command is a little different in this regard.

```
$ less file.ext
```

your explicit instructions. You'll know you have "less" in front of you if the last line of your screen either shows the file's name or just a colon (":") that waits to receive orders from you.
Hitting SPACE will scroll one page forward, "b" will scroll one page backward, and "q" will simply quit the "less" program.

Making Your Life Easier on the Command Line

There's a handful of little tricks that make your life a lot easier while working with the command line.

TAB Key

Whenever you're entering file names (including paths to a file or directory), the TAB key comes in very handy. It autocompletes what you've written, which reduces typos very efficiently. For example, when you want to switch to a different directory, you can either type every component of the path yourself:

```
$ cd ~/projects/acmedesign/documentation/
```

Or you make use of the TAB key (try this yourself!):

```
$ cd ~/pr[TAB]ojects/ac[TAB]medesign/doc[TAB]umentation/
```

In case your typed characters are ambiguous (because "dev" could be the "development" or the "developers" folder...), the command line won't be able to autocomplete. In that case, you can hit TAB another time to get all the possible matches shown and can then type a few more characters.

The command line keeps a history of the most recent commands you executed. By pressing the ARROW UP key, you can step through the last commands you called (starting with the most recently used). ARROW DOWN will move forward in history towards the most recent call.

CTRL Key

When entering commands, pressing CTRL+A moves the caret to the beginning of the line, while CTRL+E moves it to the end of the line. Finally, not all commands are finished by simply submitting them: some require further inputs from you after hitting return. In case you should ever be stuck in the middle of a command and want to abort it, hitting CTRL+C will cancel that command. While this is safe to do in most situations, please note that aborting a command can of course leave things in an unsteady state.

‹ Best Practices

Contents

Git for Subversion Users ›

TOWER

About Us

As the makers of Tower, the best Git client for Mac and Windows, we help over 100,000 users in companies like Apple, Google, Amazon, Twitter, and Ebay get the most out of Git.

Just like with Tower, our mission with this platform is to help people become better professionals.

That's why we provide our guides, videos, and cheat sheets (about version control with Git and lots of other topics) for free.

TOWER

Website Optimization
Python and Fauna Tutorial

Ruby on Rails
Tower Git Client
Visual Studio Code
Website Optimization
Workflow of Version Control
Working with Branches in Git
Xcode